Bayesian PR Review Decisions under Hidden Defect Risk

Shivam Dhoundiyal*

Abstract

Pull requests are reviewed under uncertainty: the true defect state is hidden until later testing or human review. We study a Bayesian decision procedure that observes PR evidence, maintains a belief over hidden states, and chooses among Merge, Request Human Review, and Decline / Block. The problem is specific and narrow: given CI status, diff size, file sensitivity, and author history, decide whether to merge a PR or escalate it. We compare a simple rule-based baseline against Bayesian policies on a 30-case synthetic benchmark. Policy B reduces the human-review rate and false positives relative to the baseline, but also has lower unsafe-case recall; the results remain preliminary because the priors, likelihoods, thresholds, and costs are modeling assumptions rather than production-calibrated estimates. This project is a preprint and not a submission to IJCAI.

1 Introduction

The central problem is decision-making under hidden risk. In a repository, a pull request may look safe at review time even when it contains a significant regression, data issue, or security defect. The system therefore needs to map observed PR evidence to a belief over hidden states and then choose an action under asymmetric error costs.

We focus on one specific problem: deciding whether a pull request should be merged, sent to human review, or declined when the true defect state is unobserved at the time of action. The agent input consists of four signals: CI status, diff scope, file sensitivity, and author history. The hidden states are Safe, Minor Defect, and Major Defect. The agent maintains beliefs over these states and selects an action using a cost-sensitive decision rule.

This benchmark is intentionally narrow. Its purpose is not to claim production readiness; rather, it tests whether a Bayesian policy can behave more systematically than a simple baseline under a simulated decision problem. The project code and reproduction materials are available at https://github.com/Shivam-Dhoundiyal1/PR-Review-Agent. We use the project repository data, saved metrics, and failure analysis as the grounding evidence for the paper.

*Project code and reproduction materials: https://github.com/Shivam-Dhoundiyal1/PR-Review-Agent.

2 Related work and user discussions

Prior work on software defect prediction and pull-request risk assessment shows that code review and change metadata can help forecast risk, but such signals are noisy and context-dependent [Gousios et al., 2014; Sculley et al., 2015]. In software engineering practice, repository maintainers often rely on a mix of CI, history, and code review rather than a single deterministic rule. This motivates a Bayesian decision model that can separate uncertainty from a single hard threshold.

The design in this project was also shaped by public discussions from communities including r/AI_Agents, r/devsecops, r/learnpython, r/learnmachinelearning, and r/softwareengineer. The discussion record shows that users emphasized three practical concerns: domain-aware author trust, CI reliability as a signal rather than a binary pass/fail, and the need to keep LLM reasoning separate from the probability model. These observations were incorporated in the V1 exploratory design, but the V0 benchmark remains the core evaluation because it is the only version with an externally comparable benchmark and fixed policy definitions.

The most relevant design consequence from the discussion record is that the model should treat author experience as conditional rather than absolute, and that CI reliability should be treated as a noisy signal rather than a perfect source of truth. API-rate-limit handling was explicitly excluded from the defect model because it concerns the data-collection pipeline, not the PR defect state.

3 Agent design

We define a decision-support agent whose job is to recommend one of three actions after observing PR evidence. The agent does not know the true defect state at

decision time.

3.1 Agent input

For each pull request, the agent receives:

- CI status: Pass or Fail;
- diff scope: Small, Medium, or Large;
- file sensitivity: Low-risk or High-risk;
- author history: High trust or Low trust.

These signals are intentionally simple and summary-based, so the experiment isolates the effect of Bayesian decision-making rather than a large feature engineering pipeline.

3.2 Hidden states and beliefs

The hidden states are mutually exclusive and exhaustive:

$$S_1 = \text{Safe},$$
$$S_2 = \text{Minor Defect},$$
$$S_3 = \text{Major Defect}.$$

The posterior belief over these states is a probability vector $P(S_i \mid E)$ for observed evidence E. The belief is updated from the prior and the likelihood using Bayes' rule.

3.3 Actions and error costs

The actions are:

- Merge;
- Request Human Review;
- Decline / Block.

The project cost matrix gives a much higher penalty to merging a major defect than to a false human review or a false block. For example, the modeled cost for merging a Major Defect is 100, while requesting human review for a safe PR costs only 1. This asymmetric cost structure is what motivates a threshold placed on the posterior risk of a major defect.

The baseline is a simple rule-based comparator: block failed CI, request review for high-sensitivity or large changes, and merge the rest.

4 Probability model and decision rule

The probability model is defined over three hidden states and four evidence variables. The prior probabilities are set as benchmark assumptions:

$$P(S_1) = 0.80, \quad P(S_2) = 0.15, \quad P(S_3) = 0.05, \tag{1}$$

with $\sum_{i=1}^{3} P(S_i) = 1.0$.

The likelihood terms are contained in a table that maps each evidence signal to a conditional probability under each hidden state. For a single evidence vector $E = \{E_1, E_2, E_3, E_4\}$, we assume conditional independence across evidence given the hidden state, giving

$$P(E \mid S_i) = \prod_{k=1}^{4} P(E_k \mid S_i). \tag{2}$$

Table 1: V0 benchmark results on 30 synthetic PR cases. Precision and recall are computed against the binary unsafe-vs-safe target; total decision cost uses the project cost matrix.

Policy	Precision	Recall	FP	FN	Human review rate	Total cost
Baseline	77.27%	85.00%	5	3	53.33%	56
Policy A	100.00%	40.00%	0	12	16.67%	439
Policy B	93.33%	70.00%	1	6	33.33%	142

The posterior belief is then

$$P(S_i \mid E) = \frac{P(E \mid S_i)P(S_i)}{\sum_{j=1}^{3} P(E \mid S_j)P(S_j)}. \tag{3}$$

The policy compares the posterior major-defect risk $P(S_3 \mid E)$ against thresholds. Policy A is defined as:

$$\text{Action}(E) = \begin{cases} \text{Merge}, & P(S_3 \mid E) < 0.08, \\ \text{Request Human Review}, & 0.08 \le P(S_3 \mid E) \le 0.45, \\ \text{Decline / Block}, & P(S_3 \mid E) > 0.45. \end{cases} \tag{4}$$

Policy B uses more conservative thresholds: merge below 2%, review between 2% and 20%, and block above 20%. The project records these thresholds as policy comparisons rather than externally validated production defaults.

5 Test method

We evaluate the agent on a synthetic benchmark of 30 labeled pull requests. The benchmark is stored in the project data files and is designed to test whether a Bayesian cost-sensitive decision rule can outperform a simple rule-based baseline. The hidden label is used only after the policy chooses an action; the decision is made without direct access to the hidden state.

The benchmark uses the following evaluation logic:

- an actual unsafe PR is defined as either Minor Defect or Major Defect;
- an unsafe prediction is defined as either Request Human Review or Decline / Block;
- precision, recall, false positives, and false negatives are computed on this binary target;
- total decision cost is computed using the modeled cost matrix.

This setup is intentionally synthetic; it is not a production history of merged PRs. It is best interpreted as a controlled test of decision logic and cost sensitivity rather than as a claim about a real repository.

6 Results

Table 1 summarizes the V0 benchmark results for the baseline and the two Bayesian policies.

Several conclusions follow from this result table. Policy A has zero false positives in the benchmark, but it also has twelve false negatives, placing a large cost burden on missed unsafe PRs. Policy B offers a more balanced trade-off: it retains high recall, keeps false positives low, and reduces the human-review load relative to

Table 2: Representative failure examples from the synthetic benchmark.

Case	Policy	Hidden state	Cost
TC-17	Policy A	Major Defect	100
TC-03	Policy A	Major Defect	100
TC-23	Policy A	Major Defect	100
TC-30	Policy A	Major Defect	100
TC-14	Policy B	Major Defect	5

the baseline while still being more cautious than Policy A. The baseline is the cheapest in this synthetic setting, but it does so by making more aggressive human review calls. The project therefore treats the benchmark as a comparison of decision trade-offs rather than a claim that any single policy is globally optimal.

7 Failure analysis

The project contains five high-cost failure examples in the saved failure analysis. A representative subset is enough to illustrate the key weakness: the model can underestimate risk when a PR has passing CI and a strong author history, but touches a sensitive path or a high-impact component.

The highest-cost failure is an automatic merge of a Major Defect. This occurs when passing CI and a high-trust author reduce the posterior enough to fall below the merge threshold despite a risky sensitive path. The same issue appears in TC-03, TC-23, and TC-30, where a large sensitive change is under-escalated. Policy B also misses TC-14 by requesting human review instead of blocking a Major Defect, which is less costly than a merge but still indicates that a fixed threshold can miss high-impact cases. These examples motivate a future design that distinguishes security, infrastructure, and data-destructive risk more explicitly.

8 Limitations, ethics, and human control

This work has several important limitations. First, the priors and likelihoods are benchmark assumptions, not empirically calibrated estimates from real PR histories. Second, the hidden labels are synthetic and are inherited from the controlled benchmark rather than independently collected production labels. Third, the evidence variables are intentionally coarse, and the conditional-independence assumption may be unrealistic in real repositories. These concerns are not a reason to reject the project as a benchmark, but they do constrain the claims we can make.

Ethically, the system is designed as a triage and risk-warning aid, not as an autonomous deployment authority. We do not claim that the agent is safe for autonomous use in production. The repository maintainer or designated reviewer remains the decision owner and retains the authority to override the recommendation. This addresses the key operational concern: the tool should reduce review load and highlight risk, not replace human accountability.

AI-use statement. We used AI coding tools to support drafting, sanity checks, and citation lookup. The final paper text, equations, and benchmark numbers were checked against the repository files and saved results before inclusion in the manuscript.

Contribution statement. This project was completed as team work. Contributions were distributed as follows: problem selection, public discussions, agent design, software development, tests, citation checks, preprint sections, and PDF checks.

9 Conclusion and new questions

This paper studies a Bayesian PR review agent under hidden defect risk. The problem is narrow but realistic: the agent observes limited evidence, maintains a belief over hidden defect states, and selects a cost-sensitive action. The benchmark suggests that a cautious Bayesian policy can substantially lower false positives compared with a simple rule-based baseline, but it also confirms that an uncalibrated prior or a coarse risk model can still miss major defects.

The next step is not deployment, but calibration. The project should collect independently labeled historical PRs, estimate priors and likelihoods from real data, and examine whether the model remains robust under alternative explanations such as flaky CI, unrelated change churn, or domain mismatch. The key open question is whether a real-world PR dataset produces calibrated risk estimates that remain useful for human review and release decisions.

10 References

References

[Gousios et al., 2014] Georgios Gousios, Margaret-Anne Storey, and Alberto Bacchelli. Work practices and challenges in pull-based development: The contributor's perspective. In Proceedings of the 36th International Conference on Software Engineering, pages 285–296. ACM, 2014.

[Sculley et al., 2015] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chai, Christopher Cotton, Sara Fiedler, Stephan Tian, et al. Hidden technical debt in machine learning systems. In Advances in Neural Information Processing Systems, volume 28, 2015.